Table of Contents

Cross Platform Mobile Development **1**

Terminology Introduction **1**

Widgets 1
Stateless Widgets 1
Stateful Widgets 2
Scaffold 2
MaterialApp 2

Dart Introduction **4**

Dart Datatypes 4
Dart Basics 5
Dart Class Basics 9

Cross Platform Mobile Development

Flutter is one of the leading cross platform mobile development platforms. Major competitors are ReactNative, Ionic, Xamarin and others. Flutter enables development for Android, iOS and the web.

Flutter offers a rich cohesive UI and a code-once approach for development of standard features for iOS and Android.

Disadvantages of Flutter include larger app sizes and sometimes a need to implement specific custom features on native platforms. However, most cross platform frameworks also require the implementation of some platform specific code. I have personally found Flutter to be easier to use when coding for multiple platforms when compared to ReactNative and Xamarin. I enjoy Flutter's very friendly debugging interface especially. Understandably, others may have a different preference depending on the goals of the apps you build.

Terminology Introduction

Widgets

Widgets are basically UI components in Flutter. A widget can be used to create a text label, a button or even a screen.

Stateless Widgets

A stateless widget in Flutter is a UI component that never changes once it is built. For example, the stateless widget could be used to create a screen, text, an icon or a button. The stateless widget does not store data that can update the interface while the app is running. In order for the widget to change the UI, Flutter has to rebuild it by giving it new data.

Stateful Widgets

A stateful widget in Flutter is a piece of UI that can change in appearance once it is built. The stateful widget could be used to create a screen, text, an icon or a button. The interface can respond to reflect changes of state variables that are defined and managed at the class level.

Scaffold

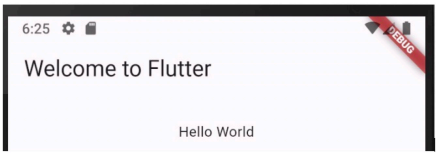
The Scaffold widget is a class which defines a standard design template for widgets to manage formatting.

MaterialApp

You will notice Flutter apps are initialized with an instance of **MaterialApp** from the **main()** function. MaterialApp sets up your app to use **Google's Material Design**. Material Design provides a standard look for Android apps: app bars, buttons, themes, navigation, etc.

Example 1: Flutter Hello World

This example demonstrates how to create a stateless hello world app. When run, Hello World appears in the emulator.



Here is the code:

```dart
import 'package:flutter/material.dart';

// The program starts here.
void main() {
  runApp( MyApp());
}

// This widget is static or unchanging.
class MyApp extends StatelessWidget {
  const MyApp({ super.key });
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(// home is the default route.
        appBar: AppBar(
          title: const Text('Welcome to Flutter'),
        ),
        body: const Center(
          child: Text('Hello World'),
        ),
      )
    );
  }
}
```

Exercise 1 (1 mark)

What two sibling widgets are included with the Scaffold widget in Example 1?

AppBar & Center

Exercise 2 (1 mark)

What two properties are included with the Scaffold widget in Example 1?

appBar and body or title and child

Exercise 3 (1 mark)

What type of class does the MyApp class inherit in Example 1?

StatelessWidget

Exercise 4 (3 mark)

MaterialApp enables consistent Material Design.

In Example 1, *home* is the default route.

Stateless widgets do not change once they are built.

Dart Introduction

Dart is a modern object-oriented programming language created by Google. Its main use is as the language behind Flutter, which allows developers to build apps for Android, iOS, web, and desktop from a single codebase. Dart looks and feels a lot like Java, C#, or JavaScript, making it easy to learn if you already know those languages. It is strongly typed, meaning every variable has a type such as **`int`**, **`double`**, or **`String`**.

Dart Datatypes

Dart has an assortment of datatypes that are familiar to most developers. Dart also provides a wide range of options to enforce how variables are defined both either for strict safety or for loose flexibility. The table below shows common data types that you will likely will encounter often.

Data type	Small description	Example

`bool`	Logical true/false.	`bool ok = true;`
		`if (ready && a > 0) {`
		`/* ... */`
		`}`

`int`	64-bit whole number.	`int count = 42;`
`double`	64-bit floating-point.	`double pi = 3.14;`
`var`	Type inferred at compile time; **not** dynamic.	`var n = 123; // infers int`
`dynamic`	Type can change.	`dynamic x = 1;` `x = 'hi'; // ok`
`List<T>`	Ordered, indexable collection of `T`.	`List<int> nums = [1, 2, 3];`
`enum`	Finite set of named values.	`enum Traffic{ red, yellow, green2 }` `var t = Traffic.red;`
`Object`	Root non-nullable type of all objects.	`Object o = 'hi';` `o = 3; // still ok`

`final`	Single-assignment variable: must be set once (at runtime) and then cannot be reassigned. The object it points to may still be mutable.	`final int port = 8080;`
		`final names = ['a', 'b'];// infers List<String>`
		`names.add('c'); // ok`
		`names = []; // Error. No reassign.`

Data type	Small description	Example

`T?` *nullable type* e.g. `String?`, `int?`	Value may be null; access requires null-aware handling (??, ?., !).	`String? name; // null`
		`print(name ?? 'guest'); // 'guest'`
		`int? x = null;`
		`print(x?.isEven); // (OK)`

`late`	Declares variable that is initialized later, but guarantees it will be assigned before use. Useful when initialization can't happen at declaration.	`late String name;` `void init() { name = "Pat"; }`

Dart Basics

Example 2: Dart Basics

The example demonstrates basic variable declaration and loop syntax in dart. When you run it, this output appears in the console.

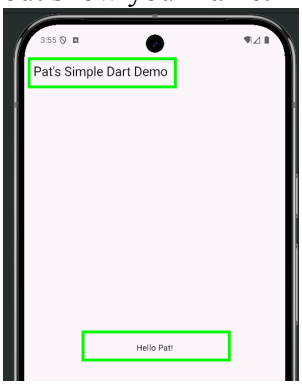
```
flutter: Hello Dart!
flutter: a + b = 8
flutter:

flutter: fruit: apple
flutter: fruit: banana
flutter: fruit: cherry
flutter:

flutter: 0: apple
flutter: 1: banana
flutter: 2: cherry
flutter: Name is: guest
```

To build this example, in a new or existing Flutter app, replace the contents of main.dart with this code.

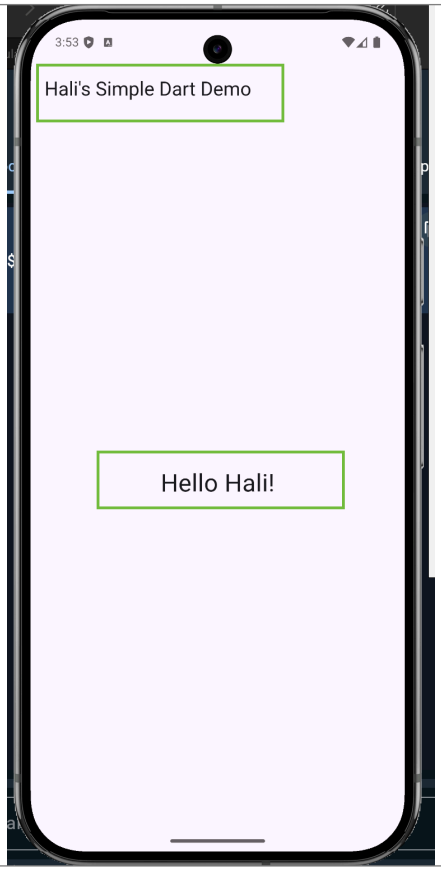
```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Simple Dart Demo',
      home: const HomeScreen(),
      debugShowCheckedModeBanner: false,
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  int getSum(int a, int b) {
    int answer = a + b;
    return answer;
  }

  void runExamples() {
    // simple dart code examples:
    print("Hello Dart!");

    int a = 5;
    int b = 3;
    int sum = getSum(a, b);
    print("a + b = ${sum}");

    print("\n");
    List<String> fruits = ["apple", "banana", "cherry"];
    for (var f in fruits) {
      print("fruit: $f");
    }

    print("\n");
    for(int i=0; i<fruits.length; i++) {
      print(i.toString() + ": " + fruits[i]);
    }

    // This is a nullable string which can store a string or nothing at all.
```

Exercise 5 (2 marks)

Modify Example 2 to show your name in the title and text area. It should look like the following but show your name.

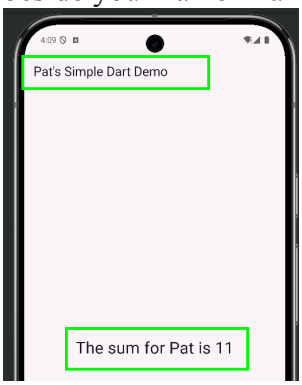


Show a screenshot of the output in your mobile emulator (not web) after making the change with your name in it.

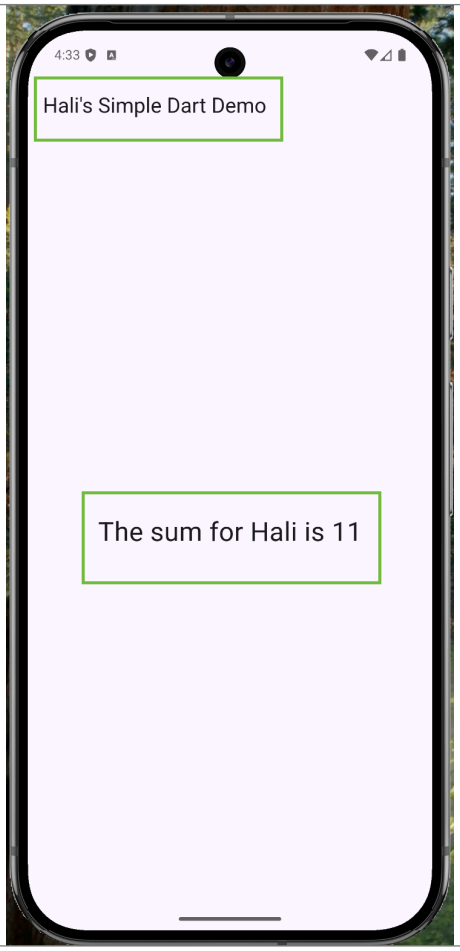


Exercise 6 (2 marks)

Modify Example 2 to dynamically show the sum that is retrieved from the getSum() method beside your name in a manner similar to this screenshot:



Show a screenshot of your output with your first name here:



Show the code for the widget block in your HomeScreen here:

```
55
56      @override
57      Widget build(BuildContext context) {
58        int sum = getSum(6, 5);
59        return Scaffold(
60          appBar: AppBar(title: const Text("Hali's Simple Dart Demo")),
61          body: Center(
62            child: Text("The sum for Hali is $sum", style: const TextStyle(fontSize: 28),),
63          ), Center
64        ); Scaffold
65      }
66    }
```

Dart Class Basics

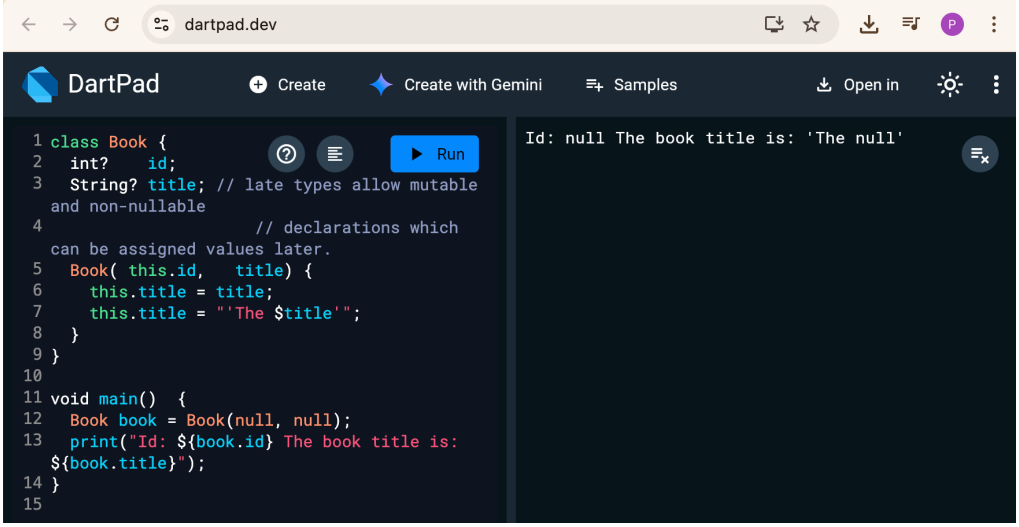
Nullable Types

Example 3: Nullable Types

This example shows how nullable types can be enabled for class variables by placing a question mark after the variable declaration.

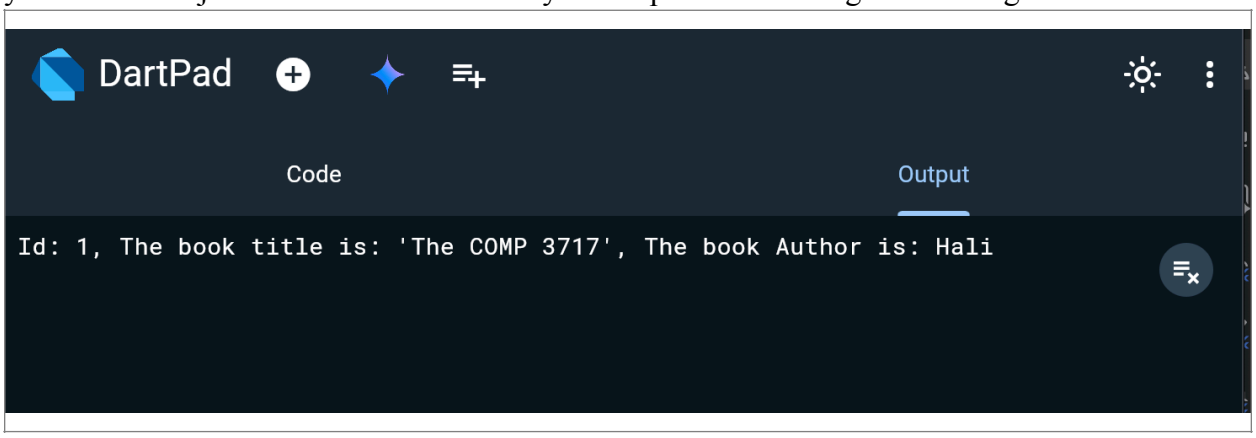
```
class Book {
  int?    id;
  String? title; // late types allow mutable and non-nullable
                     // declarations which can be assigned values later.
  Book( this.id,   title) {
    this.title = title;
    this.title = "'The $title'";
  }
}

void main()  {
  Book book = Book(null, null);
  print("Id: ${book.id} The book title is: ${book.title}");
}
```

To run this example, place your code in https://dartpad.dev/ and press the **Run** button.



Exercise 7 (1 mark)

Modify the Book class in Example 3 so it includes a nullable authorFname property. Then inside the main() function, create a second Book object which is initialized with non-null values for id and title. This second Book object must be initialized with your name. Print out the contents of your second object. Show a screenshot of your output after making these changes here:



Show your revised version of Example 3 after making these changes.

DartPad

Code | Output

Run

```
class Book {
  int?    id;
  String? title; // late types allow mutable and non-nullable
                     // declarations which can be assigned values later.
  String? authorFname; // new nullable propert
  Book( this.id,   title) {
    this.title = title;
    this.title = "'The $title'";
  }
}

void main()  {
  Book book = Book(null, null);

  // Second object
  Book book1 = Book(1, "COMP 3717");
  book1.authorFname ="Hali";
  print("Id: ${book1.id}, The book title is: ${book1.title}, The book Author is: ${book1.authorFname}");
}
```

Exercise 8 (1 mark)

Modify the Book class in Example 3 so id and title are not nullable. Show your code changes here:

```
class Book {
  int id;
  String title; // late types allow mutable and non-nullable
                    // declarations which can be assigned values later.
  String? authorFname; // new nullable propert
  Book( this.id,   title) {
    this.title = title;
    this.title = "'The $title'";
  }
}

void main()  {
  Book book = Book(null, null);

  // Second object
  Book book1 = Book(1, "COMP 3717");
  book1.authorFname ="Hali";
  print("Id: ${book1.id}, The book title is: ${book1.title}, The book Author is: ${book1.authorFname}");
}
```

What error occurs after making these changes:

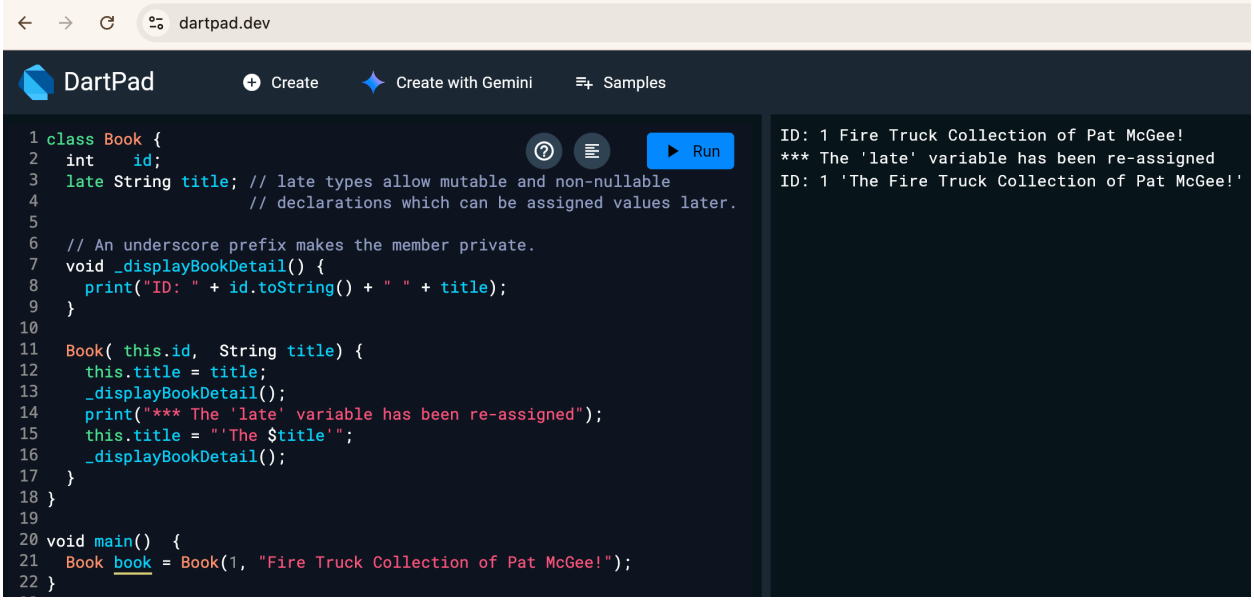
```
Error: The argument type 'Null' can't be assigned to the parameter type 'int'.
  Book book = Book(null, null);

Error: Field 'title' should be initialized because its type 'String' doesn't allow null.
  String title;
```

Example 4: Late Types

Late Types

Late types enable non-nullable and mutable declarations for variables which can be assigned values later. To run this example, place your code in https://dartpad.dev/ and press the **Run** button.

When using DartPad, the code appears on the left and the output is displayed at the right.

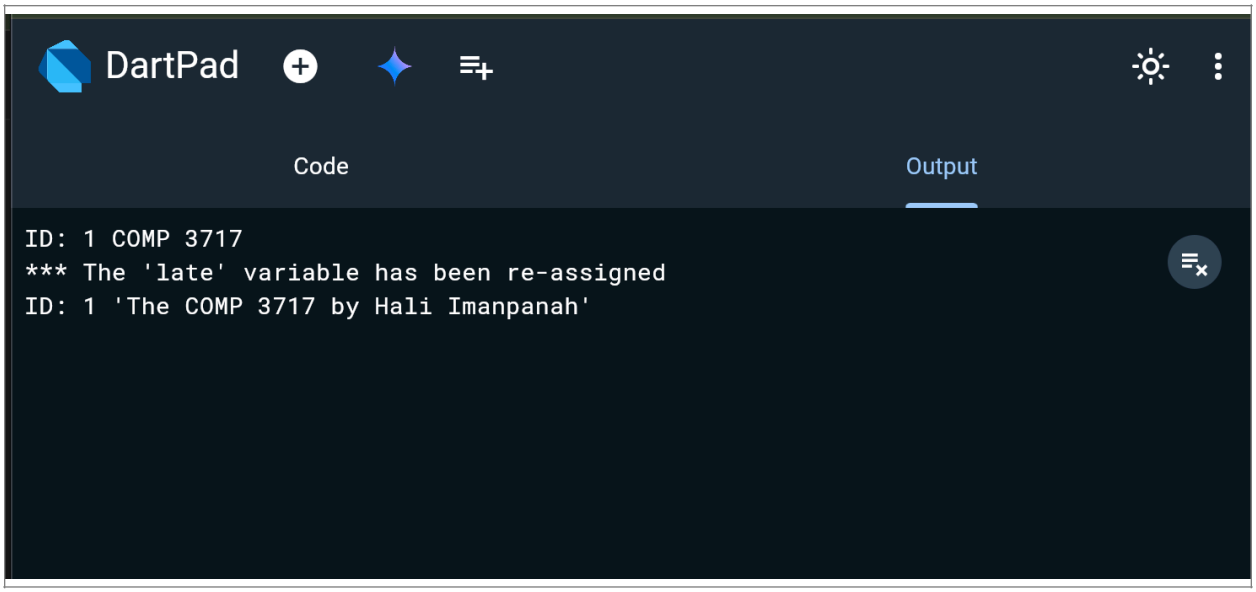


Here is the code for this example.

```
class Book {
  int    id;
  late String title; // late types allow mutable and non-nullable
                     // declarations which can be assigned values later.

  // An underscore prefix makes the member private.
  void _displayBookDetail() {
    print("ID: " + id.toString() + " " + title);
  }

  Book( this.id,  String title) {
    this.title = title;
    _displayBookDetail();
    print("*** The 'late' variable has been re-assigned");
    this.title = "'The $title'";
    _displayBookDetail();
  }
}

void main()  {
  Book book = Book(1, "Fire Truck Collection of Pat McGee!");
}
```

Exercise 9 (1 mark)

Run this code for Example 4 but show your first and last name and change the book title. Show a screenshot of the output in DartPad after making these changes here:



Named Parameters

Named parameters provide a more strict and descriptive way to enforce how objects are defined. To enable named parameters, class level variables are included within curly braces in the constructor. Then when an object of the class is declared, labelled parameters must be included.

Example 5: Named parameters

This example shows the named parameters in the constructor header and the labelled parameters when the object of the class is instantiated.

```
class Book {
  int     id;
  String  title;

  // declarations which can be assigned values later.
  Book( { required this.id,   required this.title} ) {
    this.title = title;
    this.id = id;
  }
}

void main()  {
  Book book = Book(id:5, title:"Go Dogs Go!");
  print("Id: ${book.id} The book title is: ${book.title}");
}
```

Exercise 10 (1 mark)

Modify Example 5 by adding a third named parameter called authorFName to the Book class. Make adjustments so the authorFName is also initialized when creating an object of the Book class. Also show the authorFName along with the id and title. Show your revised code here:

DartPad

Code | Output

Run

```
class Book {
  int     id;
  String  title;
  String authorFname; // New property

  // declarations which can be assigned values later.
  Book( { required this.id,   required this.title,    required this.authorFname,} ) {
    this.title = title;
    this.id = id;
    this.authorFname = authorFname;
  }
}

void main()  {
  Book book = Book(id:5, title:"Go Dogs Go!", authorFname: "Hali");
  print("Id: ${book.id}, The book title is: ${book.title}, The book author is: ${book.authorFname}");
}
```

Show a screenshot of your program output here after making these changes:

Example 6: Named Parameter Shorthand

Named parameters simplify object instantiation because the parameters are actually initialized in the constructor. Example 5 can actually be reduced to:

```
class Book {
  int     id;
  String  title;

  // declarations which can be assigned values later.
  Book( { required this.id,   required this.title} );
}

void main()  {
  Book book = Book(id:5, title:"Go Dogs Go!");
  print("Id: ${book.id} The book title is: ${book.title}");
}
```

Exercise 11 (1 mark)

Show a functional simplified version of your code for Exercise 11 after eliminating the body of the constructor. Test the code to ensure that it works. Show your code here:

DartPad

Code | Output

Run

```
class Book {
  int     id;
  String  title;
  String authorFName;

  // declarations which can be assigned values later.
  Book( { required this.id,   required this.title,   required this.authorFName} );
}

void main()  {
  Book book = Book(id:5, title:"Go Dogs Go!", authorFName: "Hali");
  print("Id: ${book.id}, The book title is: ${book.title}, The book author is: ${book.authorFName}");
}
```